

Universidad Autónoma de Madrid
Escuela Politécnica Superior

PROYECTO: APRENDIZAJE AUTOMÁTICO III

Grado en Ciencia e Ingeniería de Datos

Autor: Iván Domínguez Hernández
Autor: Lucas Miranda López
Tutor: Alberto Suárez González

30 de abril de 2026

MODELOS DE DIFUSIÓN PARA IA GENERATIVA

Autor: Iván Domínguez Hernández

Autor: Lucas Miranda López

Tutor: Alberto Suárez González

Departamento de Aprendizaje Automático
Escuela Politécnica Superior
Universidad Autónoma de Madrid

30 de abril de 2026

Resumen

[TODO: Resumen en español, 250–500 palabras. Cubrir:

- Problema (IA generativa de imágenes mediante modelos de difusión).
- Sistema desarrollado: librería `diffusion_lib` con procesos VE/VP, samplers EM/PC/PF-ODE, schedules lineal/coseno, métricas BPD/FID/IS y generación controlable (CFG e imputación).
- Resultados principales: FID/IS sobre los datasets entrenados.
- Conclusiones de alto nivel.

]

Palabras clave: modelos de difusión, IA generativa, score matching, SDE, U-Net, FID, BPD.

Abstract

[TODO: Resumen en inglés, 250–500 palabras. Traducción del español.]

Keywords: diffusion models, generative AI, score matching, SDE, U-Net, FID, BPD.

Contents

Resumen	iii
Abstract	v
1 Introducción	1
1.1 Alcance	1
1.2 Estructura del documento	1
2 Estado del Arte	3
2.1 Introducción al problema	3
2.1.1 Objetivo	3
2.1.2 Estimación de densidad	3
2.2 Modelos de difusión	3
2.2.1 Proceso forward (inyección de ruido)	3
2.2.2 Score matching y función de pérdida	3
2.2.3 Proceso backward (generación)	3
2.2.4 Variance Exploding (VE)	3
2.2.5 Variance Preserving (VP)	3
2.2.6 Ecuación de Fokker-Planck	3
2.2.7 Integración numérica de la SDE inversa: samplers	4
2.3 Generación controlable	4
2.3.1 El problema de la generación condicionada	4
2.3.2 Classifier guidance	4
2.3.3 Classifier-free guidance	4
2.3.4 Función de pérdida	4
2.3.5 Planteamiento de las SDEs	4
2.3.6 SDE backward condicionada	4
2.3.7 Imputación	4
2.4 Métricas de calidad para las imágenes generadas	4
2.4.1 Negative log-likelihood (NLL)	4
2.4.2 Bits per dimension (BPD)	5
2.4.3 Fréchet Inception Distance (FID)	5
2.4.4 Inception Score (IS)	5
2.5 Fenómeno del double descent	5
3 Desarrollo de Software y Diseño	7
3.1 Análisis de Requisitos	7
3.1.1 Requisitos funcionales	7
3.1.2 Requisitos no funcionales	8
3.2 Casos de Uso	8
3.3 Arquitectura y Diseño	9
3.3.1 Estructura del paquete	9

3.3.2	Diagrama de clases	9
3.3.3	Diagrama de secuencia (entrenamiento y muestreo)	9
3.4	Validación y Calidad del Software	10
3.4.1	Tests automatizados	10
3.4.2	Integración continua	10
3.4.3	Estilo y documentación	10
3.4.4	Licencia	10
3.4.5	Implicaciones éticas	10
3.5	Planificación del Proyecto	10
4	Resultados	11
4.1	Comparación de procesos: VE vs VP	11
4.2	Comparación de schedules: lineal vs coseno (sobre VP)	11
4.3	Comparación de samplers: EM vs PC vs PF-ODE	11
4.4	Generación controlable	12
4.4.1	Class-conditional con CFG	12
4.4.2	Imputación	12
4.5	Resultados sobre dataset color	12
4.6	Análisis de rendimiento	12
5	Conclusiones	13
	Bibliografía	14
A	Autoencoders, espacios latentes y manifold hypothesis	17
B	U-Net	19
C	Movimiento Browniano y proceso de Wiener	21
D	De <i>denoising score matching</i> individual al objetivo combinado multi-escala	23
E	Gaussiana multivariada, divergencia Kullback–Leibler y ELBO	25
F	Integración numérica de la SDE inversa: detalles	27
G	Comparativa de métricas	29

Chapter 1

Introducción

1.1 Alcance

El presente proyecto tiene como objetivo fundamental la construcción de un sistema de Inteligencia Artificial generativa, altamente configurable, capaz de sintetizar imágenes tanto en blanco y negro como en color.

Para lograr este objetivo, el alcance del sistema desarrollado abarca la implementación de los siguientes componentes técnicos:

- **Procesos de difusión para la inyección de ruido:** modelos *Variance Exploding* (VE) y *Variance Preserving* (VP).
- **Muestreadores (samplers):** integrador estocástico Euler–Maruyama, esquema Predictor–Corrector y *Probability Flow ODE* determinista.
- **Programaciones de ruido (noise schedules):** decaimiento lineal y de tipo coseno.
- **Métricas de evaluación:** *Negative Log-Likelihood* (NLL), *Bits Per Dimension* (BPD), *Fréchet Inception Distance* (FID) e *Inception Score* (IS).
- **Generación controlable:** muestreo condicionado a clases (CFG) e imputación de imágenes parcialmente observadas.

1.2 Estructura del documento

[TODO: Reescribir cuando esté la versión final. Texto de partida:]

En el **Capítulo 2** se presenta el Estado del Arte, con la formulación matemática de los modelos de difusión, los samplers, las *noise schedules*, las métricas de calidad y el problema de la generación condicionada.

El **Capítulo 3** aborda el Desarrollo de Software y Diseño, detallando el análisis de requisitos, los casos de uso, la arquitectura del paquete `diffusion_lib` y la planificación temporal.

En el **Capítulo 4** se exponen los Resultados, evaluando el rendimiento, la robustez y la calidad gráfica del sistema mediante comparativas y métricas cuantitativas.

Finalmente, el **Capítulo 5** recoge las Conclusiones del trabajo, y el documento se cierra con la bibliografía y los apéndices técnicos.

Chapter 2

Estado del Arte

2.1 Introducción al problema

[TODO: Trasvasar de tu PDF actual la subsección "Introducción al problema", "Objetivo" y "Estimación de densidad". Ya están bien redactadas.]

2.1.1 Objetivo

[TODO: Texto existente.]

2.1.2 Estimación de densidad

[TODO: Texto existente. Arreglar el cite [2, Section 3.1] si compila mal.]

2.2 Modelos de difusión

2.2.1 Proceso forward (inyección de ruido)

[TODO: Texto existente.]

2.2.2 Score matching y función de pérdida

[TODO: Texto existente. *Recordar:* añadir frase corta presentando la U-Net como aproximador del score y remitir al apéndice correspondiente.]

2.2.3 Proceso backward (generación)

[TODO: Texto existente.]

2.2.4 Variance Exploding (VE)

[TODO: Texto existente. Al final, añadir párrafo de ventajas y limitaciones (lo pide el enunciado).]

2.2.5 Variance Preserving (VP)

ARREGLAR: el cosine schedule debe ir con cuadrado: $f(t) = \cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right)$.]

2.2.6 Ecuación de Fokker-Planck

[TODO: Texto existente.]

2.2.7 Integración numérica de la SDE inversa: samplers

Cubre los tres samplers (EM, PC, PF-ODE) que son requisito 2 del enunciado.]

2.3 Generación controlable

[TODO: RENOMBRAR de "Modelos de procesos de difusión para generación condicionada" a "Generación controlable" para que cuadre con el enunciado (apartado 5).]

2.3.1 El problema de la generación condicionada

[TODO: Texto existente.]

2.3.2 Classifier guidance

[TODO: Texto existente.]

2.3.3 Classifier-free guidance

[TODO: Texto existente.]

2.3.4 Función de pérdida

[TODO: Texto existente.]

2.3.5 Planteamiento de las SDEs

[TODO: Texto existente.]

2.3.6 SDE backward condicionada

[TODO: Texto existente.]

2.3.7 Imputación

[TODO: Subsección NUEVA. Tienes el archivo redactado en `subseccion_imputacion.tex`. Pegar aquí. Cubre el requisito 5b del enunciado.]

2.4 Métricas de calidad para las imágenes generadas

2.4.1 Negative log-likelihood (NLL)

[TODO: Texto existente. ARREGLAR:

- Eliminar la frase "(que para este problema pueden ser las mismas que para train)" — la NLL en train sobreestima.
- Nombrar explícitamente la *dequantization* uniforme (Theis et al. 2016) en la definición $y = x + u$.
- La expresión del ELBO debe ser *desigualdad* ($\geq$), no igualdad.

]

2.4.2 Bits per dimension (BPD)

[TODO: Texto existente.]

2.4.3 Fréchet Inception Distance (FID)

[TODO: Texto existente.]

2.4.4 Inception Score (IS)

[TODO: Texto existente. ARREGLAR: las referencias a las ecuaciones (2.28) y (2.29) están rotas, no existen. Renumerar con \ref.]

2.5 Fenómeno del double descent

[TODO: O bien desarrollar como subsección (definición Belkin/Nakkiran 2019, relación con la capacidad de la U-Net y el tamaño del dataset) o bien eliminar de aquí y mover como observación empírica al Capítulo de Conclusiones, citando los datos de §5.]

Chapter 3

Desarrollo de Software y Diseño

3.1 Análisis de Requisitos

Los siguientes requisitos se han definido de manera específica para permitir la ejecución de tests automatizados que demuestren su cumplimiento.

3.1.1 Requisitos funcionales

[TODO: Lista numerada con referencias al §2. Ejemplos:

- RF-01: El sistema debe implementar el proceso de difusión VE descrito en §2.2.4.
- RF-02: El sistema debe implementar el proceso de difusión VP descrito en §2.2.5.
- RF-03: El sistema debe permitir seleccionar entre los procesos VE y VP mediante un parámetro de configuración.
- RF-04: El sistema debe implementar el sampler Euler–Maruyama.
- RF-05: El sistema debe implementar el sampler Predictor–Corrector con SNR objetivo configurable.
- RF-06: El sistema debe implementar el sampler Probability Flow ODE.
- RF-07: El sistema debe implementar las schedules lineal y coseno.
- RF-08: El sistema debe calcular las métricas BPD, FID e IS sobre un conjunto de muestras.
- RF-09: El sistema debe permitir la generación condicionada por clase mediante CFG.
- RF-10: El sistema debe permitir la imputación de imágenes parcialmente observadas mediante una máscara binaria.
- RF-11: El sistema debe soportar imágenes en blanco y negro (1 canal) y en color (3 canales).

]

3.1.2 Requisitos no funcionales

Entorno y herramientas. [TODO: Versiones exactas. Mirar `.python-version` y `uv.lock`:

- Python: [TODO: X.Y]
- PyTorch: [TODO: X.Y]
- Gestor de dependencias: `uv`
- Otras: NumPy, scipy, matplotlib, torchvision, scikit-image, etc.

]

Hardware. [TODO: GPU usada (modelo, VRAM), CPU, RAM. Indicar mínimos para inferencia y para entrenamiento por separado.]

Rendimiento. [TODO: Especificaciones cuantitativas verificables, p.ej.: "Una imagen 28×28 debe generarse en menos de 2 s con el sampler Euler–Maruyama y $N = 500$ pasos en una GPU NVIDIA RTX 3060". Cada uno debe corresponderse con una entrada de la tabla de Resultados §4.]

Compatibilidad. [TODO: Sistemas operativos soportados, formatos de checkpoint (`.pth`), interoperabilidad con otros frameworks.]

Estilo y documentación. [TODO: PEP-8 verificado con `ruff/black`, type hints, docstrings estilo NumPy, generación automática con Sphinx/mkdocs.]

Metodología de desarrollo. [TODO: Git/GitHub, ramas *feature*, pull requests, revisión cruzada, CI con GitHub Actions.]

3.2 Casos de Uso

[TODO: Describir los casos de uso principales, cada uno enlazado a un notebook real del proyecto. Ejemplos:

- CU-01 — Generación incondicional: usuario entrena un modelo VP sobre MNIST y muestrea N imágenes nuevas. Notebook: `project_AAIII_teamCode.ipynb`.
- CU-02 — Generación condicionada por clase: usuario carga modelo CFG entrenado sobre SVHN y genera imágenes de la clase 7. Notebook: `cfg_diffusion_models_SVHN.ipynb`.
- CU-03 — Imputación: usuario carga imagen parcialmente observada y completa los píxeles faltantes.
- CU-04 — Comparativa de samplers: usuario evalúa FID de EM, PC y PF-ODE sobre el mismo modelo.
- CU-05 — Cálculo de BPD: usuario evalúa BPD sobre un conjunto de test.

]

3.3 Arquitectura y Diseño

3.3.1 Estructura del paquete

El sistema se organiza como un paquete Python instalable con la siguiente estructura de módulos:

Listing 3.1: Estructura del paquete `diffusion_lib`

```
diffusion_lib/
    __init__.py
    model.py                # U-Net (score network)
    processes/              # Procesos de difusi n
        base.py            # BaseProcess (ABC)
        ve.py              # Variance Exploding
        vp.py              # Variance Preserving
    samplers/               # Integradores de la SDE inversa
        base.py            # BaseSampler (ABC)
        euler_maruyama.py
        predictor_corrector.py
        probability_flow_ode.py
        imputation.py
    schedules/              # Noise schedules
        base.py            # BaseSchedule (ABC)
        linear.py
        cosine.py
    metrics/                # M tricas de calidad
        bpd.py             # Bits per dimension
        fid_is.py          # FID e IS sobre InceptionV3
```

3.3.2 Diagrama de clases

[TODO: Insertar UML simple. Sugerencia: usar PlantUML o draw.io y exportar a PDF. Mostrar:

- `BaseProcess` $\leftarrow$ `VE`, `VP`
- `BaseSampler` $\leftarrow$ `EulerMaruyama`, `PredictorCorrector`, `ProbabilityFlowODE`, `Imputation`
- `BaseSchedule` $\leftarrow$ `Linear`, `Cosine`
- Relaciones de composición (`Sampler` usa `Process`, `Process` usa `Schedule`).

]

3.3.3 Diagrama de secuencia (entrenamiento y muestreo)

[TODO: Opcional pero recomendado. Mostrar el flujo:

1. Entrenamiento: `dataset` $\rightarrow$ `Process.add_noise` $\rightarrow$ `U-Net.forward` $\rightarrow$ `loss` $\rightarrow$ `optimizer`.
2. Muestreo: `prior` $\rightarrow$ `Sampler.step` (loop) $\rightarrow$ `Process.reverse_drift` $\rightarrow$ `U-Net` $\rightarrow$ `imagen`.

]

3.4 Validación y Calidad del Software

3.4.1 Tests automatizados

[TODO: Estrategia: `pytest` con tests unitarios por módulo (`test_processes.py`, `test_samplers.py`, etc.) y al menos un test de integración. Indicar cobertura.]

3.4.2 Integración continua

[TODO: GitHub Actions: workflow con steps de `ruff` check, `pytest`, build de docs.]

3.4.3 Estilo y documentación

[TODO: PEP-8, `black`, `ruff`, `docstrings NumPy`, `Sphinx` para auto-documentación de la API.]

3.4.4 Licencia

[TODO: Decidir y justificar: MIT (permisiva) / Apache-2.0 (incluye patentes) / GPL (copyleft). Para un proyecto académico/open-source suele recomendarse MIT.]

3.4.5 Implicaciones éticas

[TODO: Riesgos de modelos generativos: deepfakes, sesgos heredados de los datasets (MNIST/SVHN/gatos no son problemáticos, pero conviene mencionar el riesgo en datasets faciales o con sesgo demográfico), coste energético del entrenamiento.]

3.5 Planificación del Proyecto

[TODO: Diagrama de Gantt. Fases sugeridas:

1. Investigación bibliográfica y revisión teórica (semanas 1–3).
2. Implementación de procesos VE/VP y entrenamiento básico (semanas 3–5).
3. Implementación de samplers (semanas 4–6).
4. Implementación de schedules y métricas (semanas 6–7).
5. Generación condicionada e imputación (semanas 7–9).
6. Generación de resultados y comparativas (semanas 9–11).
7. Redacción del informe (semanas 8–12, paralelo).

Recomendado: usar la plantilla de TikZ `pgfgantt` o exportar desde una hoja de cálculo.]

Chapter 4

Resultados

[**TODO: Introducción breve del capítulo: qué se compara, sobre qué datasets, con qué métricas, condiciones de evaluación reproducibles (seed, hardware, número de muestras).**]

4.1 Comparación de procesos: VE vs VP

Incluir:

- Galería visual lado a lado (8–16 muestras por proceso).
- Tabla con FID, IS y BPD por proceso.
- Curvas de loss vs época superpuestas.
- Discusión: estabilidad de entrenamiento, calibración de $\sigma_{\max}$ (VE) vs $\beta_{\max}$ (VP).

]

4.2 Comparación de schedules: lineal vs coseno (sobre VP)

Incluir:

- Galería visual.
- Trayectoria forward (figura tipo Nichol & Dhariwal pero *generada con tu código*, no copiada).
- Tabla FID/IS/BPD.
- Discusión: por qué coseno conserva señal hasta más tarde y ayuda en baja resolución.

]

4.3 Comparación de samplers: EM vs PC vs PF-ODE

Setup: mismo modelo entrenado, misma seed, distinto sampler.

Incluir:

- Galería visual lado a lado.
- Tabla con FID, tiempo por imagen y #evals de red por muestra.

- **Curva calidad vs N** (FID en función de $N \in \{10, 25, 50, 100, 250, 500, 1000\}$). Esta gráfica es la que mejor muestra la diferencia entre samplers.
- Demostración del determinismo del PF-ODE: dos corridas con la misma seed dan resultado idéntico; con EM o PC, distintas.

]

4.4 Generación controlable

4.4.1 Class-conditional con CFG

Sweep del peso de guidance $w \in \{0, 1, 3, 5, 10\}$ para mostrar el trade-off fidelidad-clase vs diversidad.]

4.4.2 Imputación

Tres filas por ejemplo:

- Imagen original.
- Máscara aplicada.
- Reconstrucción (varias muestras para mostrar diversidad).

Probar al menos tres tipos de máscara: bandas horizontales, cuadrado central, ruido aleatorio.]

4.5 Resultados sobre dataset color

Sobre los gatos / SVHN:

- Galería de muestras de los checkpoints (5, 50, 300 épocas...).
- Métricas FID/IS para cada checkpoint.
- Discusión: relación con el fenómeno del double descent (si se mantiene esa observación en §2).

]

4.6 Análisis de rendimiento

Tabla:

- Filas: combinaciones (proceso $\times$ sampler $\times N$).
- Columnas: segundos/imagen, memoria pico GPU, #evals red, cumple-RNF (sí/no).

]

Chapter 5

Conclusiones

Resumen del trabajo realizado. Recapitular brevemente los objetivos del Cap. 1 y relacionar cada uno con lo entregado.

Logros.

- Sistema modular `diffusion_lib` que cubre los 5 puntos del enunciado.
- Implementación reproducible de VE/VP, EM/PC/PF-ODE, schedules lineal/coseno, métricas BPD/FID/IS, generación controlable.
- Validación cuantitativa con métricas estándar de la literatura.

Limitaciones.

- Datasets de baja resolución (MNIST, SVHN, gatos 32×32).
- U-Net sin las mejoras de las arquitecturas más recientes (DiT, EDM).
- Sin paralelismo de datos / multi-GPU.
- La calidad sobre imágenes naturales sigue lejos de los SOTA.

Trabajo futuro.

- Extender a resoluciones mayores (64×64 , 128×128) con arquitectura adaptada.
- Integradores de orden superior para PF-ODE (Heun, RK4, DPM-Solver).
- Métricas alternativas: CLIP-FID, Precision & Recall.
- Modelos latentes (Latent Diffusion).

Reflexión personal. Lecciones aprendidas, dificultades técnicas encontradas, conexión con otras asignaturas del grado.]

Bibliography

Appendix A

Autoencoders, espacios latentes y manifold hypothesis

[TODO: Texto existente del apéndice A.]

Appendix B

U-Net

[TODO: Texto existente. Recordar: añadir `\ref{ap:unet}` desde §2.2.2 para que el lector encuentre la arquitectura.]

Appendix C

Movimiento Browniano y proceso de Wiener

[TODO: Texto existente.]

Appendix D

De *denoising score matching* individual al objetivo combinado multi-escala

[TODO: Texto existente.]

Appendix E

Gaussiana multivariada, divergencia Kullback–Leibler y ELBO

[TODO: Texto existente.]

Appendix F

Integración numérica de la SDE inversa: detalles

[TODO: Versión *larga* de los samplers (la versión compacta va al cuerpo principal en §2.2.7). Aquí: derivaciones completas, análisis de convergencia, integradores de orden superior.]

Appendix G

Comparativa de métricas

[TODO: Texto existente del apéndice de comparativa.]